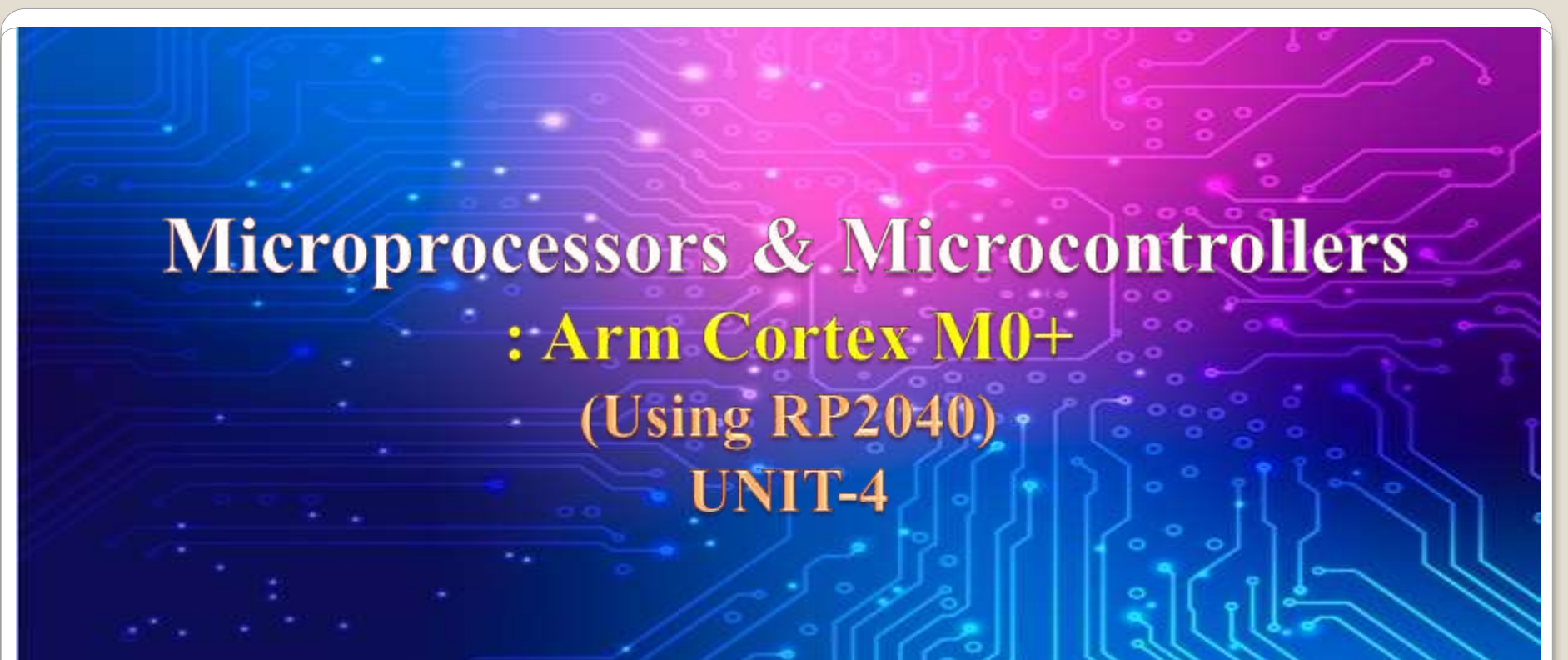


Embedded System and Microcontroller



Microprocessors & Microcontrollers

: Arm Cortex M0+

(Using RP2040)

UNIT-4

ldr pseudo, adc, sub and mul instructions

Mouli Sankaran



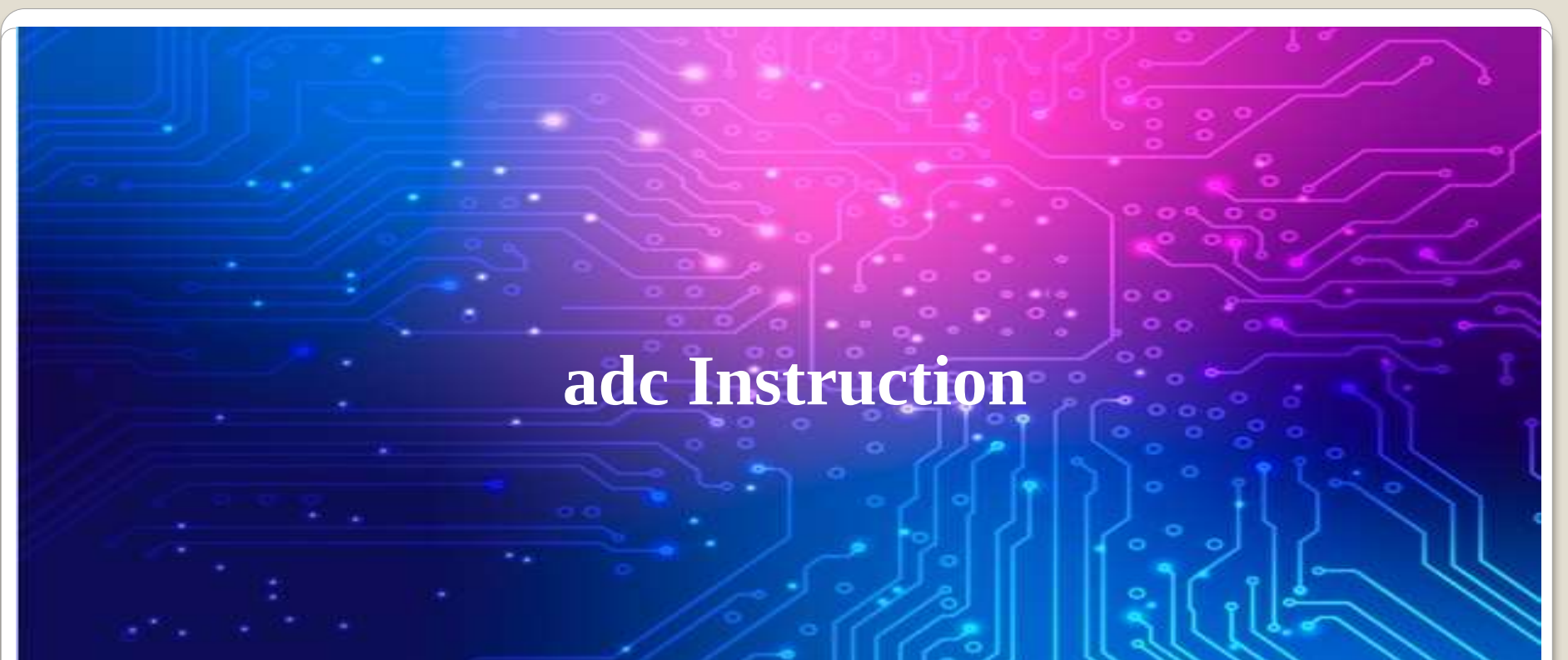
ldr Rd, =label Instruction

ldr Rd,=label pseudo-instruction

- LDR Rd,=label can load any **32-bit numeric** value into a register.
- It also accepts PC-relative expressions such as labels, and labels with offsets.
- The assembler converts an LDR Rd,=label pseudo-instruction by:
- Placing the 32-bit contents mentioned as label in a literal pool (a portion of memory embedded in the code to hold constant values).
- Generating a PC-relative LDR instruction that reads the 32-bit contents from the literal pool, for example:

```
LDR rn [pc, #offset_to_literal_pool]
        ; load register n with one word
        ; from the address [pc + offset]
```

Ref: [Load addresses to a register using LDR Rd, =label](#)



adc Instruction

Technical Reference Manual of Cortex-M0+ (Ref 3)

Table 3-1 Cortex-M0+ instruction summary

Operation	Description	Assembler	Cycles
Move	8-bit immediate	MOV _S Rd, #<imm>	1
	Lo to Lo	MOV _S Rd, Rm	1
	Any to Any	MOV Rd, Rm	1
	Any to PC	MOV PC, Rm	2
Add	3-bit immediate	ADD _S Rd, Rn, #<imm>	1
	All registers Lo	ADD _S Rd, Rn, Rm	1
	Any to Any	ADD Rd, Rd, Rm	1
	Any to PC	ADD PC, PC, Rm	2
	8-bit immediate	ADD _S Rd, Rd, #<imm>	1
	With carry	ADCS Rd, Rd, Rm	1
	Immediate to SP	ADD SP, SP, #<imm>	1
	Form address from SP	ADD Rd, SP, #<imm>	1
	Form address from PC	ADR Rd, <label>	1

Note: Ignore the S suffix in the above instructions. They are not valid for RP2040. Only the instructions without the S suffix are to be used.

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Bits	Name	Function
[31]	N	Negative flag.
[30]	Z	Zero flag.
[29]	C	Carry or borrow flag.
[28]	V	Overflow flag.
[27:0]	-	Reserved.

- *Application Program Status Register (APSR).*
- *Interrupt Program Status Register (IPSR).*
- *Execution Program Status Register (EPSR).*

	31	30	29	28	27	25	24	23						6	5		0
APSR	N	Z	C	V	Reserved												
IPSR	Reserved														Exception number		
EPSR	Reserved				T	Reserved											



sub and mul Instructions

Technical Reference Manual of Cortex-M0+ (Ref 3)

Subtract	Lo and Lo	SUBS Rd, Rn, Rm	1
	3-bit immediate	SUBS Rd, Rn, #<imm>	1
	8-bit immediate	SUBS Rd, Rd, #<imm>	1
	With carry	SBCS Rd, Rd, Rm	1
	Immediate from SP	SUB SP, SP, #<imm>	1
	Negate	RSBS Rd, Rn, #0	1
Multiply	Multiply	MULS Rd, Rm, Rd	1 or 32 ^a

Note: Ignore the S suffix in the above instructions. They are not valid for RP2040. Only the instructions without the S suffix are to be used.



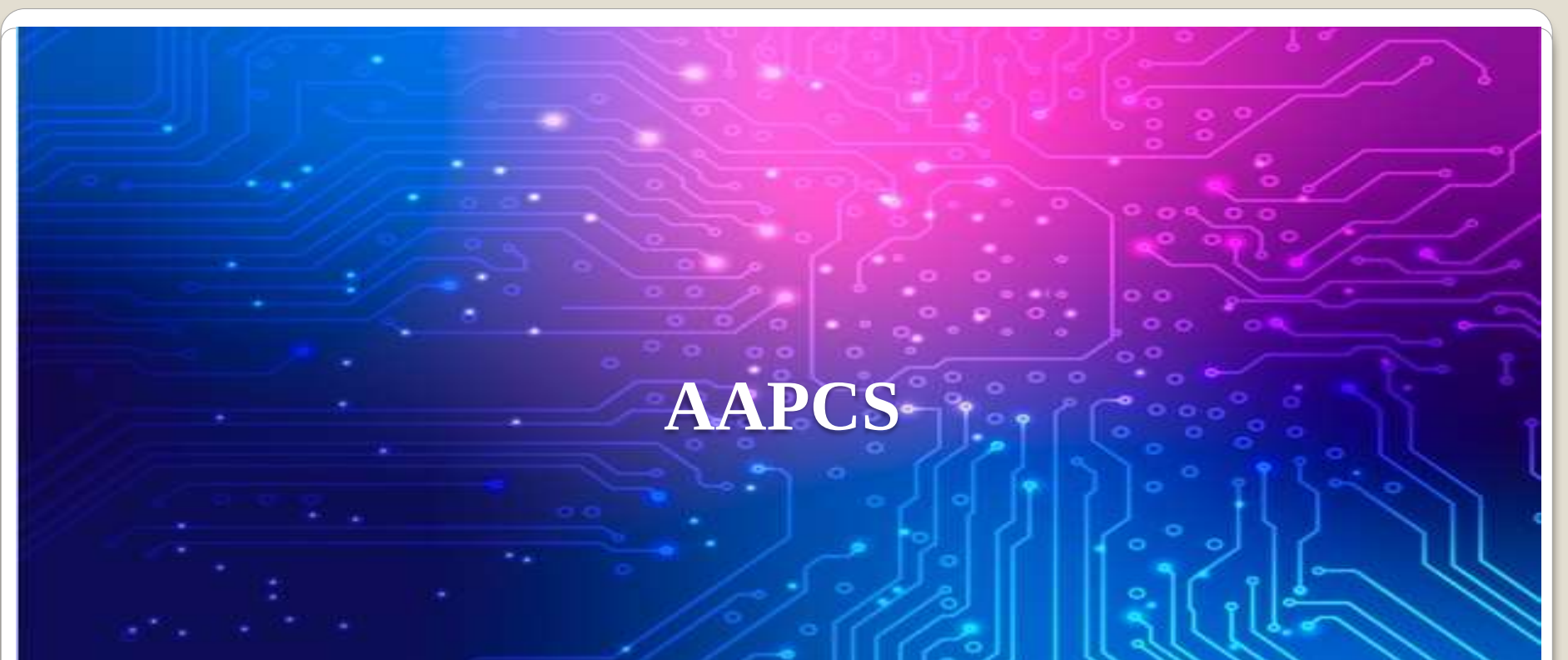
Demo of Asm Program

prog_lec_2_3_2.ino and main_asm.S



Microprocessors & Microcontrollers **: Arm Cortex M0+** **(Using RP2040)**

Application Binary Interface (ABI) and AAPCS



AAPCS

Procedure Call Standard for the Arm[®] Architecture (AAPCS)

[Article on AAPCS by Chis Shore](#)

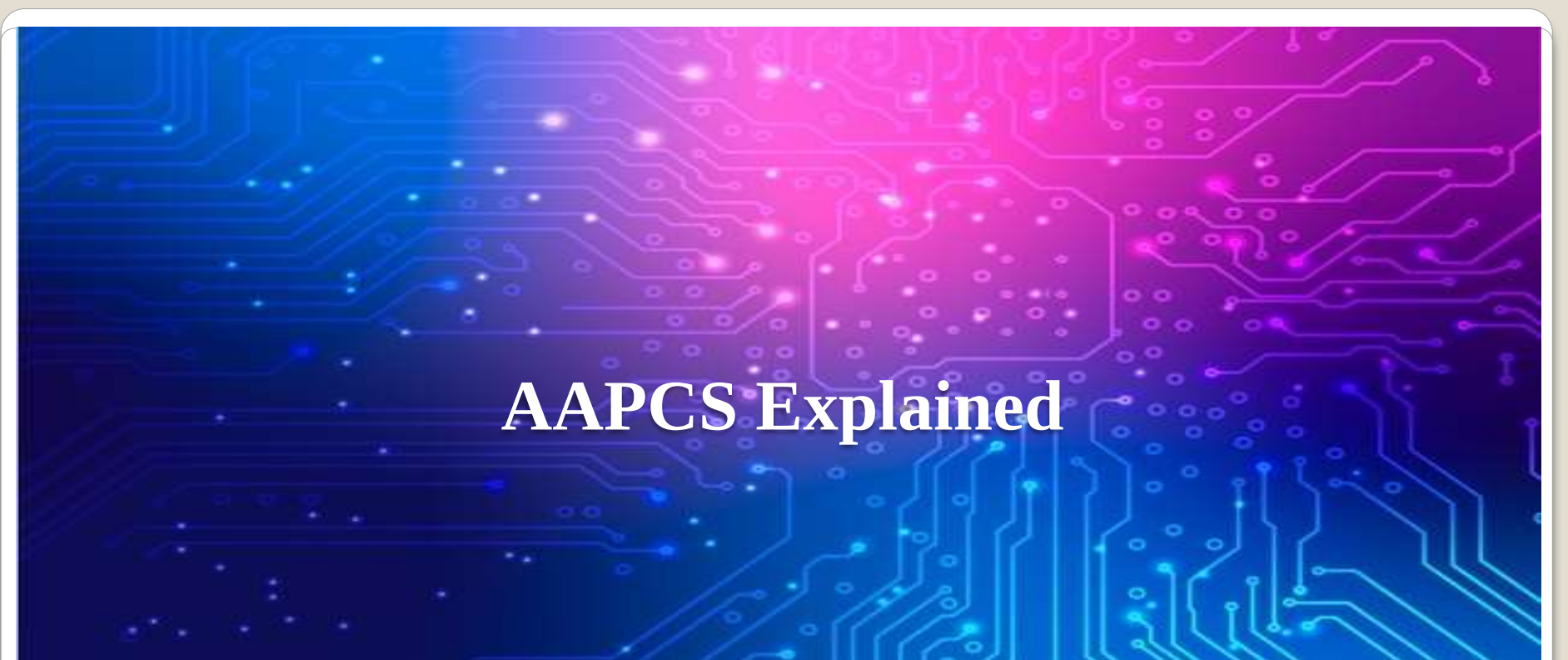
Application Binary Interface (ABI)

- ABI for ARM Architecture is a collection of standards, some open and some specific to the ARM architecture
- It regulates the **interoperation** of **binary** files and development **tools** in a spectrum of ARM architecture-based execution environments
 - From **bare metal** to major **operating systems** such as ARM Linux
- ABI for ARM Architecture includes
 - ARM Architecture Procedure Call Standard (**AAPCS**)
 - ARM **ELF** (Executable and Linkable Format)
 - ARM **DWARF** (Debugging With Attribute Record Format)
 - The Base Platform ABI (**BPABI**)
 - C++ ABI, Exception Handling ABI, Run-time ABI, C library ABI, etc.

Procedure Call Standard for the Arm® Architecture (AAPCS)



- The AAPCS defines how subroutines can be separately written, separately compiled, and separately assembled to work together.
- It describes a contract between a **calling routine** and a **called routine** that defines:
 - Obligations on the caller to create a program state in which the called routine may start to execute.
 - Obligations on the called routine to preserve the program state of the caller across the call.
 - The rights of the called routine to alter the program state of its caller.



AAPCS Explained

Fundamental Data Types

Type Class	Machine Type	Byte size	Byte alignment
Integral	Unsigned byte	1	1
	Signed byte	1	1
	Unsigned half-word	2	2
	Signed half-word	2	2
	Unsigned word	4	4
	Signed word	4	4
	Unsigned double-word	8	8
	Signed double-word	8	8

- Unsigned Byte is for Char types

Fundamental Data Types

Type Class	Machine Type	Byte size	Byte alignment
Floating Point	Half precision	2	2
	Single precision (IEEE 754)	4	4
	Double precision (IEEE 754)	8	8
Containerized vector	64-bit vector	8	8
	128-bit vector	16	8
Pointer	Data pointer	4	4
	Code pointer	4	4

- Floating Point **Half precision** is **16 bit** format
- Pointer arithmetic should be unsigned

Core Registers

Register	Synonym	Special	Role in the procedure call standard
r15		PC	The Program Counter.
r14		LR	The Link Register.
r13		SP	The Stack Pointer.

- **LR** is for holding the **return address** on subroutine call
- **Stack** is a contiguous area of memory that is used for storage of
 - **Local variables**
 - For passing **additional arguments** to subroutines when there are **insufficient argument registers** available
- Stack implementation is **full-descending**, with the current extent of the stack held in the register SP (r13)
- Stack may in general have both a **base** and a **limit**
 - Mostly applications may not be able to estimate the limit accurately
- Stack may have a fixed size or be dynamically extendable
 - By adjusting the stack-limit downwards

Core Registers ... contd.

Register	Synonym	Special	Role in the procedure call standard
r12		IP	The Intra-Procedure-call scratch register.
r11	v8		Variable-register 8.
r10	v7		Variable-register 7.
r9		v6 SB TR	Platform register. TR : Thread Register The meaning of this register is defined by the platform
r8	v5		Variable-register 5.
r7	v4		Variable register 4.
r6	v3		Variable register 3.
r5	v2		Variable register 2.
r4	v1		Variable register 1.

- **IP (R12)** is to hold **intermediate** values between subroutine calls
- The role of register **R9** is platform specific

Core Registers ... contd.

Register	Synonym	Special	Role in the procedure call standard
r3	a4		Argument / scratch register 4.
r2	a3		Argument / scratch register 3.
r1	a2		Argument / result / scratch register 2.
r0	a1		Argument / result / scratch register 1.

- **R0-R3** are used for passing arguments to subroutine
- Registers **R4-R8, R10** and **R11** (v1-v5, v7 and v8) are used to hold the values of a routine's **local variables**
- Of these, only **v1-v4 (R4-R7)** can be used by the whole **Thumb** instruction set, but the AAPCS does not require that Thumb code only use those registers
- A subroutine must preserve the contents of the registers **R4-R8, R10, R11** and **SP**

Larger Data Types

- A double-word sized type is passed in two consecutive registers (e.g., **R0** and **R1**, or **R2** and **R3**)
 - The contents of these registers are as if the double-word value had been loaded from memory with a **single LDM** instruction
- A **128-bit** containerized vector is passed in **four consecutive registers**
 - The contents of the registers are as if the value had been loaded from memory with a single LDM instruction

Returning Result

The following data types are returned in **R0** and or **R1** after conversion

- Half-precision FP after conversion to single precision
- Fundamental Data Types that are smaller than 4 bytes is zero- or sign-extended to a word (in **R0**)
- Word-sized Fundamental Data Type (e.g. int, float) (in **R0**)
- Double-word sized Fundamental Data Type (e.g., long long, double and 64-bit containerized vectors (in **R0** and **R1**)
- 128-bit containerized vector is returned in **R0-R3**

Returning Result ... contd.



The following data types are returned in **R0** and or **R1** after conversion

- Composite Type **not larger than 4 bytes** is returned in **R0**
- A Composite Type **larger than 4 bytes**, or whose size that cannot be determined statically by both caller and callee
 - It is stored in memory at an address passed as an **extra argument** when the function was called
 - Caller decides the location and callee stores the result in that location before returning back to Caller from the routine
- The memory to be used for the result may also be modified at any point during the function call



Microprocessors & Microcontrollers

: Arm Cortex M0+

(Using RP2040)

cmp and conditional branch instructions

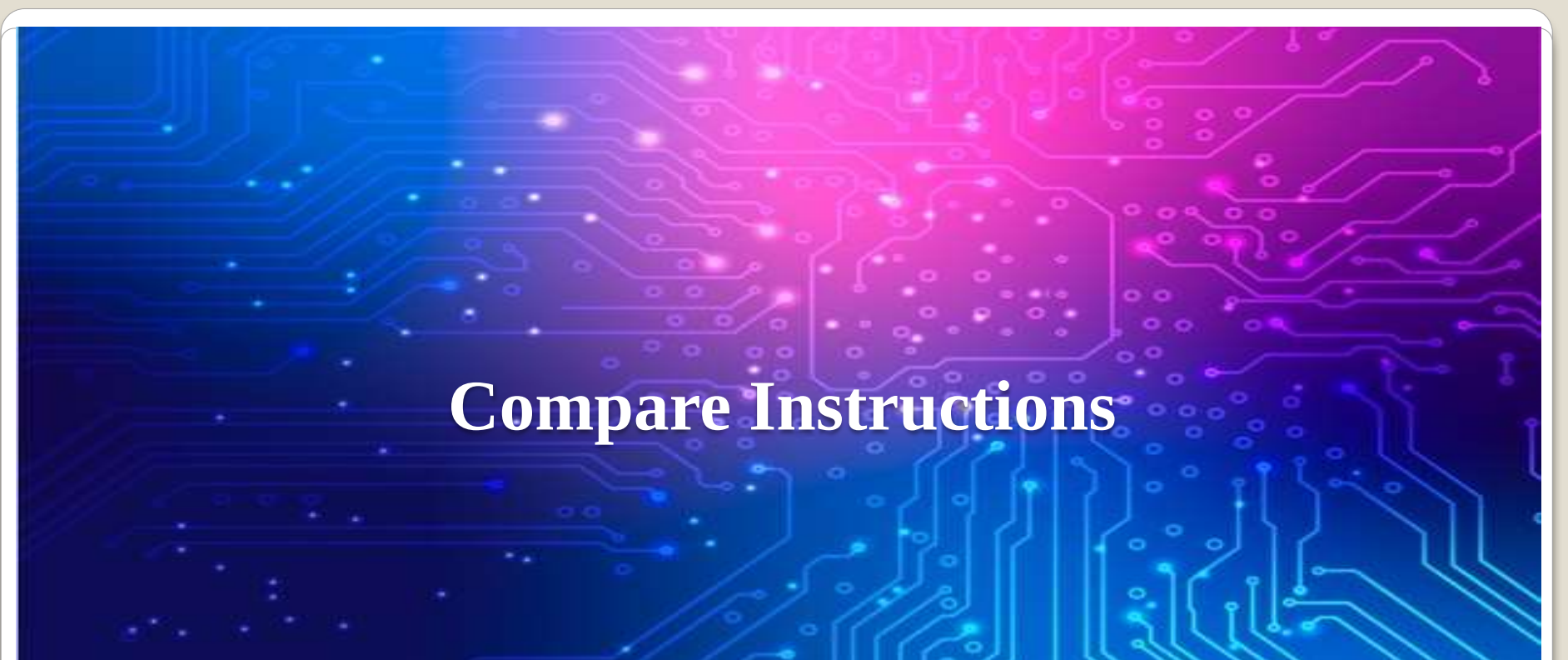
Lecture 2.4.1 Focus

- **cmp** (compare) instructions
- **Conditional branch** instructions
- Demo of **Asm** program

Ref:

Ref3_ARM-Cortex-M0+ Technical Reference Manual.pdf

Ref4_ARM-Cortex-M0+ Devices Generic User Guide.pdf



Compare Instructions

Technical Reference Manual of Cortex-M0+ (Ref 3)

Page: 26/51

Compare	Compare	CMP Rn, Rm	1
	Negative	CMN Rn, Rm	1
	Immediate	CMP Rn, #<imm>	1

- These instructions are required to implement the high level language conditional statements within the if/while/for constructs
- These compare instructions change the conditional flags by performing a subtract operation without changing the contents of the operands
- After the flags are modified, conditional branch instructions are used by checking the relevant conditional flags affected by CMP to change the control flows



Conditional branch Instructions

Technical Reference Manual of Cortex-M0+ (Ref 3)

Page: 28/51

e. 2 cycles if the branch is taken, one cycle if not-taken.

Branch	Conditional	B<cc> <label>	1 or 2 ^e
	Unconditional	B <label>	2
	With link	BL <label>	3
	With exchange	BX Rm	2
	With link and exchange	BLX Rm	2

- These instructions are required to implement the high level language conditional statements within the if/while/for constructs
- These instructions are used to implement different conditional statements of higher level language constructs that change the flow of execution
 - **CMP** instructions are used to perform the comparison and these **conditional branch** instructions to change the flow of the program

cc above refers to the predefined condition codes defined by Arm

**Let us see
them next ...**

ARM Conditions Code (cc)

Condition Code	Interpretation	Status Flag State
EQ	Equal / equals zero	Z set
NE	Not equal	Z clear
CS / HS	Carry set / unsigned higher or same	C set
CC / LO	Carry clear / unsigned lower	C clear
MI	Minus / negative	N set
PL	Plus / positive or zero	N clear
VS	Overflow	V set
VC	No overflow	V clear
HI	Unsigned higher	C set and Z clear
LS	Unsigned lower or same	C clear or Z set
GE	Signed greater than or equal	N equals V
LT	Signed less than	N is not equal to V
GT	Signed greater than	Z clear and N equals V
LE	Signed less than or equal	Z set or N is not equal to V
AL	Always (optional)	Any

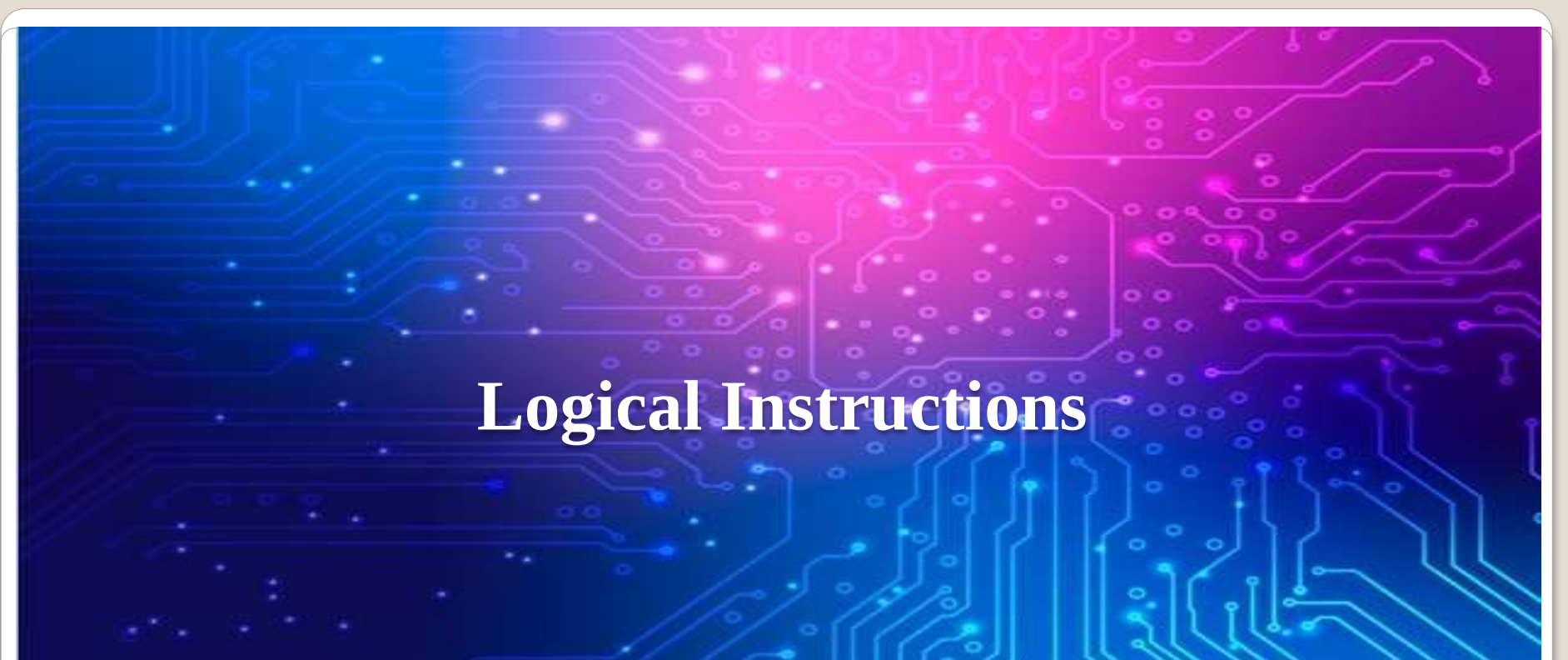
[Source link:](#)

Note: The **NZCV** flags are affected by the **CMP** instruction.



Microprocessors & Microcontrollers **: Arm Cortex M0+** **(Using RP2040)**

Logical, Shift and Rotate Instructions



Logical Instructions

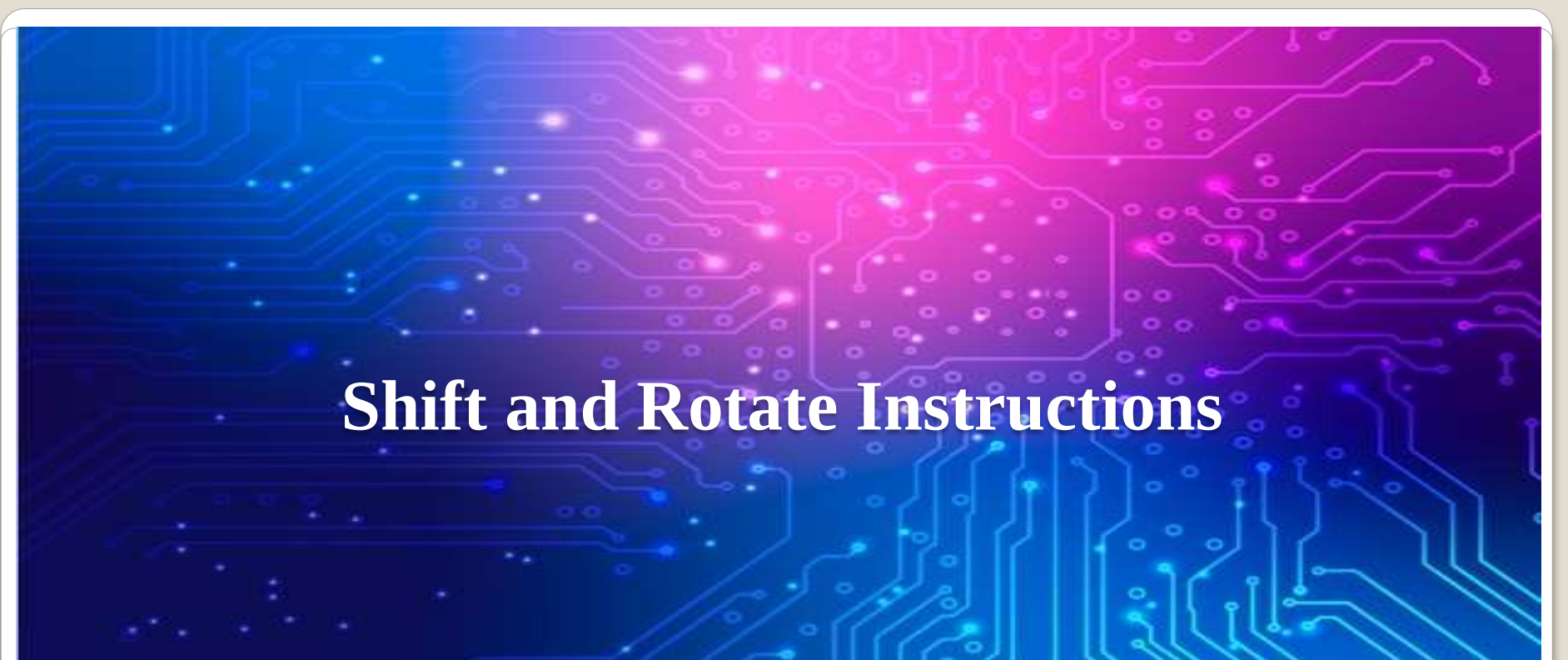
Technical Reference Manual of Cortex-M0+-(Ref 3) Logical

Page: 27/51 **Table 3-1 Cortex-M0+ instruction summary (continued)**

Operation	Description	Assembler	Cycles
Logical	AND	ANDS Rd, Rd, Rm	1
	Exclusive OR	EORS Rd, Rd, Rm	1
	OR	ORRS Rd, Rd, Rm	1
	Bit clear	BICS Rd, Rd, Rm	1
	Move NOT	MVNS Rd, Rm	1
	AND test	TST Rn, Rm	1

Note: Do not add the **S** suffix to the instructions
It is **not supported** on **RP2040** only without S suffix.
It is not supported in **Thumb 16** mode.

- The AND, EOR, and ORR instructions perform bitwise AND, exclusive OR, and inclusive OR operations on the values in Rd and Rm.
- The BIC instruction performs an AND operation on the bits in Rd with the logical negation of the corresponding bits in the value of Rm.
- In these instructions, Rd, Rn, and Rm must only specify R0-R7.
- These instructions only update the N and Z flags according to the result and do not affect the C or V flag.



Shift and Rotate Instructions

Technical Reference Manual of Cortex-M0+-(Ref 3) – Shift and Rotate

Page: 27/51 **Table 3-1 Cortex-M0+ instruction summary (continued)**

Operation	Description	Assembler	Cycles
Shift	Logical shift left by immediate	LSLS Rd, Rm, #<shift>	1
	Logical shift left by register	LSLS Rd, Rd, Rs	1
	Logical shift right by immediate	LSRS Rd, Rm, #<shift>	1
	Logical shift right by register	LSRS Rd, Rd, Rs	1
	Arithmetic shift right	ASRS Rd, Rm, #<shift>	1
	Arithmetic shift right by register	ASRS Rd, Rd, Rs	1
Rotate	Rotate right by register	RORS Rd, Rd, Rs	1

Note: Do not add the **S** suffix to the instructions
It is **not supported** on **RP2040** only without S suffix.
It is not supported in **Thumb 16** mode.

- ASR, LSL, LSR, and ROR perform an arithmetic-shift-left, logical-shift-left, logical-shift-right or a right-rotation of the bits in the register **Rm** by the number of places specified by the immediate **imm** or the value in the least-significant byte of the register specified by **Rs**.
- In these instructions, Rd, Rm, and Rs must only specify R0-R7.
- For non-immediate instructions, Rd and Rs must specify the same register.



Demo of Asm Program

prog_lec_2_4_2.ino and main_asm.S



Demo of Asm Program

prog_lec_2_4_1.ino and main_asm.S

References - 2

Ref 3

Cortex[®]-M0+
Revision: r0p1
Technical Reference Manual

Ref 4

Cortex[®]-M0+ Devices
Generic User Guide

Ref this document
for Assembly instructions

Copyright © 2012 ARM. All rights reserved.
ARM DDI 0494C (DDI171C)

ARM

Copyright © 2012 ARM. All rights reserved.
ARM DDI 0862B (DDI171B)

ARM

RP2040: A microcontroller by Raspberry Pi

RP2040 Datasheet
A microcontroller
by Raspberry Pi

Ref 6

RP2040: A microcontroller by Raspberry Pi

Raspberry Pi Pico C/C++ SDK
Libraries and tools for
C/C++ development on
RP2040 microcontrollers